Although this now works and is race-free, there's still a performance issue. Once one thread has started a `push()` operation by successfully completing the `compare_exchange_strong()` on `old_tail.ptr->data` (❺ from listing 7.15), no other thread can perform a `push()` operation. Any thread that tries will see the new value rather than `nullptr`, which will cause the `compare_exchange_strong()` call to fail and make that thread loop again. This is a busy wait, which consumes CPU cycles without achieving anything. Consequently, this is effectively a lock. The first `push()` call blocks other threads until it has completed, so *this code is no longer lock-free*. Not only that, but whereas the operating system can give priority to the thread that holds the lock on a mutex if there are blocked threads, it can't do so in this case, so the blocked threads will waste CPU cycles until the first thread is done. This calls for the next trick from the lock-free bag of tricks: the waiting thread can help the thread that's doing the `push()`.

Making the Queue Lock-Free by Helping out Another Thread

In order to restore the lock-free property of the code, you need to find a way for a waiting thread to make progress even if the thread doing the `push()` is stalled. One way to do this is to help the stalled thread by doing its work for it.

In this case, you know exactly what needs to be done: the `next` pointer on the tail node needs to be set to a new dummy node, and then the `tail` pointer itself must be updated. The thing about dummy nodes is that they're all equivalent, so it doesn't matter if you use the dummy node created by the thread that successfully pushed the data or the dummy node from one of the threads that's waiting to push. If you make the `next` pointer in a node atomic, you can then use `compare_exchange_strong()` to set the pointer. Once the `next` pointer is set, you can then use a `compare_exchange_weak()` loop to set the `tail` while ensuring that it's still referencing the same original node. If it isn't, someone else has updated it, and you can stop trying and loop again. This requires a minor change to `pop()` as well in order to load the `next` pointer; this is shown in the following listing.